

Lab 16 — Priority Inversion and Mutexes

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2 hrs
Proves	priority inheritance, measured
Prerequisites	Labs 2 and 8 (tasks, priorities, kernel objects).
Beyond the board	Nothing. The whole experiment is internal to the scheduler.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **\$16.1 objective**, **\$16.2 parts and wiring**, **\$16.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept first, then **Phase 0 configure**, **Phase 1 the complete code**, walkthrough, **Phase 2 build and run**, **Phase 3 prove it**, then worked extensions. Every file is given in full. **Nothing is left undefined and nothing is left for you to figure out.** Follow it top to bottom.

16.1 Objective and TM4C bridge

Reproduce priority inversion deliberately, measure how long it delays a high-priority task, then fix it with a mutex and measure again. You will end with two numbers you can quote in an interview.

TM4C BRIDGE

Bare metal has no scheduler, so it has no priority inversion. What it does have is the NVIC's interrupt priority levels, and the same idea appears there: a low-priority interrupt holding a resource a high-priority interrupt needs. Bare metal, you solve that by disabling interrupts around the critical section. With an RTOS you have tasks rather than interrupts, and the tool is a mutex with priority inheritance.

16.2 Parts and wiring

Board and USB cable. Nothing else. Everything in this lab is internal to the scheduler.

16.3 How it works

The three things you need in place first

Priority. Every FreeRTOS task has a number, and the scheduler always runs the highest-numbered task that is *ready*. Higher wins. That word "ready" is doing a lot of work — a blocked task is not ready and is skipped entirely, no matter how high its priority.

A shared resource. Something only one task may use at a time. You protect it with a lock, and any task wanting it must take the lock first.

One core. Everything here is pinned to core 0 deliberately. With two cores a medium-priority task could simply run on the other core and the effect would be masked.

The scenario

Three tasks. **LOW** (priority 3) takes the lock and holds it for 300 ms of work. **HIGH** (priority 8) wants that same lock and times how long it waits. **MED** (priority 5) never touches the lock — it just burns 500 ms of CPU at a priority *between* the other two.

1. LOW takes the lock and starts working.
2. HIGH wakes, wants the lock, finds it held, and **blocks**.
3. The scheduler runs the highest-priority *ready* task. HIGH is blocked, so that leaves MED (5) and LOW (3), and **MED outranks LOW**.
4. MED preempts LOW and runs for its full 500 ms.
5. LOW cannot run, so it cannot finish, so it cannot release the lock.
6. HIGH — the highest-priority task in the system — waits on a task two levels below it.

Note the crucial detail in step 4: **MED is not doing anything wrong**. It never touched the lock. It is simply higher priority than the lock's holder, and that is enough.

Why this is a safety problem, not a performance one

The delay is **unbounded**. Add a second medium-priority task and HIGH waits longer. There is no number you can put on the worst case, and "no bound on worst-case latency" is what a real-time system cannot tolerate.

On **Mars Pathfinder** in 1997 this happened for real. A high-priority bus-management task blocked on a lock held by a low-priority meteorological task while medium-priority communications work ran. The delay tripped a watchdog, which reset the spacecraft — repeatedly, from Mars. Engineers reproduced it on a ground replica and uploaded a fix enabling priority inheritance.



Figure 16.1. The six steps as a timeline. Read across the HIGH row: it is blocked for the whole of MED's execution, despite outranking it by three priority levels.

Priority inversion, as a timeline.

Three rows, one per task: - HIGH (priority 8): runs briefly, attempts to take the lock ("try take"), then sits in a long BLOCKED band, and only resumes at the far right. - MED (priority 5): runs straight through the middle of HIGH's blocked band. - LOW (priority 3): takes the lock at the start, is preempted by MED, and only releases the lock much later.

The inversion is visible as: HIGH waits while MED, a LOWER priority task, runs.

The fix is a mutex with priority inheritance. When HIGH blocks on the lock, LOW is temporarily boosted to HIGH's priority, so MED cannot preempt it. LOW finishes at once, releases, and HIGH runs.

This exact bug reset NASA's Mars Pathfinder in 1997, caused by a mutex configured without priority inheritance.

The fix, and why only one of the two primitives can do it

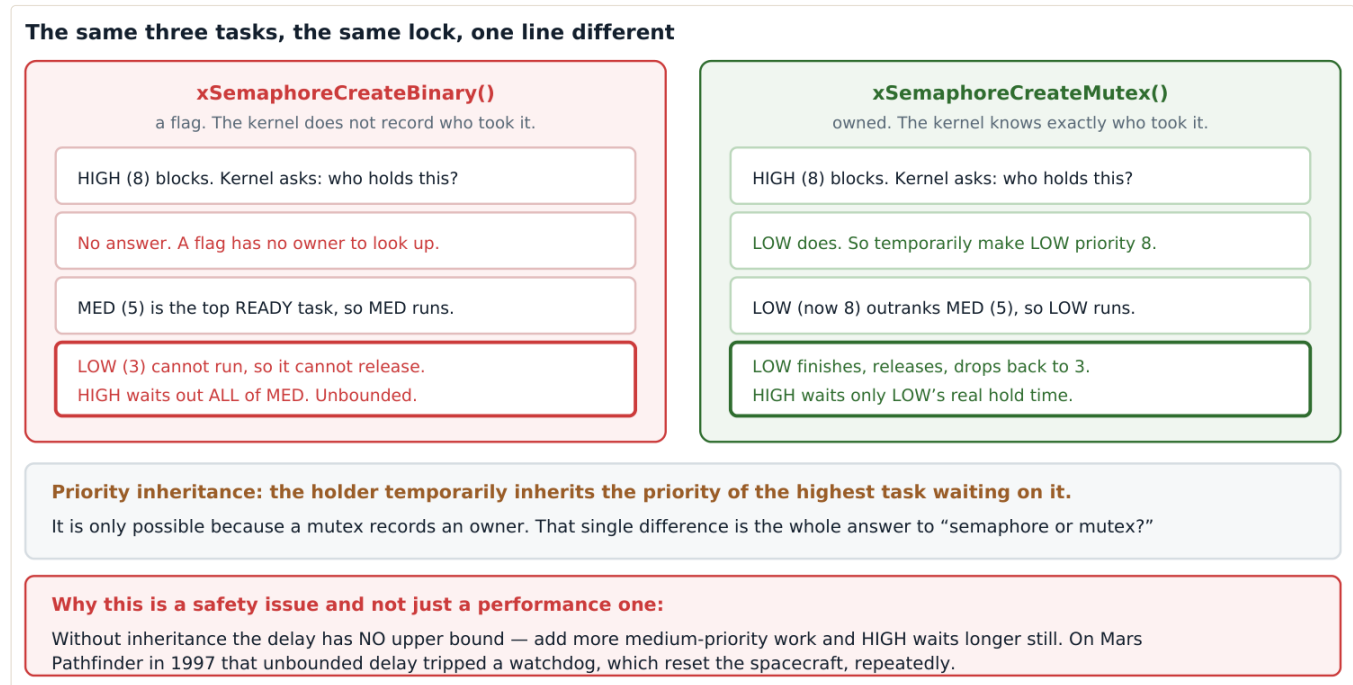


Figure 16.2. A binary semaphore has no owner, so the kernel has nobody to promote. A mutex records its owner, which is exactly what makes priority inheritance possible.

Why a binary semaphore cannot fix priority inversion but a mutex can.

Same three tasks, same lock, one line different.

With `xSemaphoreCreateBinary()` -- a flag, and the kernel does not record who took it: 1. HIGH (priority 8) blocks. The kernel asks: who holds this? 2. No answer. A flag has no owner to look up. 3. MED (priority 5) is the top READY task, so MED runs. 4. LOW (priority 3) cannot run, so it cannot release. HIGH waits out ALL of MED. Unbounded.

With `xSemaphoreCreateMutex()` -- owned, and the kernel knows exactly who took it: 1. HIGH (priority 8) blocks. The kernel asks: who holds this? 2. LOW does. So temporarily make LOW priority 8. 3. LOW (now 8) outranks MED (5), so LOW runs. 4. LOW finishes, releases, and drops back to 3. HIGH waits only LOW's real hold time.

Priority inheritance: the holder temporarily inherits the priority of the highest task waiting on it. It is only possible because a mutex records an owner. That single difference is the whole answer to "semaphore or mutex?"

Why this is a safety issue and not just a performance one: without inheritance the delay has NO upper bound. Add more medium-priority work and HIGH waits longer still. On Mars Pathfinder in 1997 that unbounded delay tripped a watchdog, which reset the spacecraft, repeatedly.

A **binary semaphore has no concept of ownership**. It is a flag: the kernel tracks whether it is taken, not *who* took it, and any task may give it back. So when HIGH blocks, the kernel has no answer to "who is holding this?"

A **mutex is owned** by whichever task took it, and only that task may release it. That ownership enables **priority inheritance**: the moment HIGH blocks on a mutex held by LOW, the kernel temporarily raises LOW to HIGH's priority. Now LOW outranks MED, runs to completion, releases, and drops back to 3.

KEY POINT

The rule, and the reason behind it

Signalling between contexts → semaphore or queue. Protecting a shared resource → mutex, always.

Plenty of people can recite that. What makes it a good interview answer is the reason: a mutex has an owner, which permits priority inheritance, which bounds the worst-case delay.

Why the experiment is driven by notifications, not delays

WARNING

This is the part earlier editions got wrong, and it is worth understanding

The obvious way to build this experiment is to give each task a different `vTaskDelay` and hope the collision lands. It does not work. With LOW delaying 1000 ms, HIGH 1050 ms and MED 1100 ms, LOW takes the lock at $t=0$ and releases at $t=300$, and HIGH first wakes at $t=1050$ — **750 ms after the lock is already free**. It never blocks, so there is nothing to measure. Simulated, that arrangement gives a wait of **0 ms in both the semaphore and the mutex run**.

Worse, the three periodic loops then drift apart by different amounts, so any later collision is scheduler-dependent rather than designed. That is the opposite of a controlled experiment.

So this lab orders the experiment explicitly with task notifications. Every step is *caused*, not timed. Run it a hundred times and it does the same thing a hundred times.

The forced sequence:

1. CTRL starts a trial by notifying LOW.
2. LOW takes the lock, then notifies HIGH. HIGH is priority 8, so it preempts *immediately*, tries the lock, and blocks. The inversion condition now exists **by construction**.
3. LOW resumes and notifies MED.
4. With a **semaphore**, MED (5) preempts LOW (3) and burns 500 ms. With a **mutex**, LOW has inherited priority 8 and MED cannot touch it.
5. LOW finishes its 300 ms and releases.
6. HIGH acquires and records its wait.

16.4 Phase 0 — configure

Two files in the **project root** (the folder holding the top-level `CMakeLists.txt`, not `main/`).

0.1 — sdkconfig.defaults

```
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
```

```
CONFIG_FREERTOS_HZ=1000
CONFIG_ESP_TASK_WDT_INIT=n
```

`CONFIG_FREERTOS_HZ=1000` raises the tick from the default 100 Hz to 1 kHz, giving 1 ms resolution instead of 10 ms. Without it your measured waits quantise to multiples of 10 ms and the numbers look artificial.

`CONFIG_ESP_TASK_WDT_INIT=n` disables the task watchdog. This lab *deliberately* spins the CPU for 300 and 500 ms at a time, which is exactly what the watchdog exists to catch. Leave it on and it fires during every trial.

0.2 — top-level CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab16_inversion)
```

No `EXTRA_COMPONENT_DIRS` here — this lab needs no network, so `protocol_examples_common` is not involved.

0.3 — main/CMakeLists.txt

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS ".")
```

0.4 — apply and verify

```
Remove-Item sdkconfig -ErrorAction SilentlyContinue
idf.py fullclean
idf.py set-target esp32s3
idf.py reconfigure
```

Then check the generated header, which is what the compiler actually sees:

```
Select-String -Path build\config\sdkconfig.h -Pattern "CONFIG_FREERTOS_HZ|CONFIG_ESP_TASK_WDT_INIT"
```

KEY POINT

Expected output

You want to see `#define CONFIG_FREERTOS_HZ 1000`. If it says 100, the defaults file was not picked up — check it is in the project root and that you deleted `sdkconfig` first. **sdkconfig.defaults is only read when no sdkconfig exists.** That single rule causes more confusion than anything else in ESP-IDF configuration.

`CONFIG_ESP_TASK_WDT_INIT` should be absent or 0. Absent is correct — a disabled boolean is not defined at all, rather than defined as 0.

16.5 Phase 1 — the complete main.c

This is the whole file. It compiles as printed. Change `USE_MUTEX` to switch between Part A and Part B; nothing else changes.

```
// =====
// Lab 16 -- Priority inversion, measured, then fixed with a mutex
// Complete file. Set USE_MUTEX to 0 for Part A, 1 for Part B.
// =====

#include <stdio.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_timer.h"
#include "esp_log.h"

// ----- knobs
#define USE_MUTEX    0          // 0 = Part A (binary semaphore), 1 = Part B (mutex)
#define CORE         0          // pin everything to ONE core, or the effect is masked
#define HOLD_MS      300        // LOW's critical section
#define MED_MS       500        // MED's CPU burn -- never touches the lock
#define TRIALS        30
#define PRIO_LOW      3
#define PRIO_MED      5
#define PRIO_HIGH     8
#define PRIO_CTRL     9        // above all three, so it always regains control

static const char *TAG = "lab16";

static SemaphoreHandle_t lock;
static TaskHandle_t h_low, h_med, h_high, h_ctrl;

static int64_t t_high_blocked;      // written by HIGH only
static int32_t results[TRIALS];    // written by HIGH only
static int     n_results;           // written by HIGH only
static bool    med_ran_during_hold; // written by MED, read by CTRL

// ----- helpers
// A deliberate CPU hog. Everywhere else in this series a busy-wait is a bug.
// Here it is REQUIRED: vTaskDelay would yield the core, no contention would
// occur, and the inversion this lab exists to demonstrate could never happen.
static void busy(int ms)
{
    int64_t end = esp_timer_get_time() + (int64_t)ms * 1000;
    while (esp_timer_get_time() < end) { }
}

// ----- tasks
static void low_task(void *arg)
{
    (void)arg;
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);    // 1. wait for CTRL

        xSemaphoreTake(lock, portMAX_DELAY);        // 2. LOW now holds the lock
        ESP_LOGI(TAG, "  LOW_LOCKED");
    }
}
```

```

        xTaskNotifyGive(h_high);           // 3. HIGH preempts HERE and blocks
        xTaskNotifyGive(h_med);           // 4. MED becomes ready

        ESP_LOGI(TAG, "  LOW_RUN");
        busy(HOLD_MS);                     // 5. the real critical section

        xSemaphoreGive(lock);
        ESP_LOGI(TAG, "  LOW_UNLOCKED");

        xTaskNotifyGive(h_ctrl);           // 6. trial over
    }
}

static void med_task(void *arg)
{
    (void)arg;
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        med_ran_during_hold = true;        // evidence for the trace check
        ESP_LOGI(TAG, "  MED_RUN");
        busy(MED_MS);                     // never touches the lock
    }
}

static void high_task(void *arg)
{
    (void)arg;
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        t_high_blocked = esp_timer_get_time(); // clock starts BEFORE the take
        ESP_LOGI(TAG, "  HIGH_BLOCKED");

        xSemaphoreTake(lock, portMAX_DELAY); // blocks until LOW releases
        int32_t waited_ms =
            (int32_t)((esp_timer_get_time() - t_high_blocked) / 1000);

        ESP_LOGI(TAG, "  HIGH_ACQUIRED after %" PRIu32 " ms", waited_ms);
        if (n_results < TRIALS) results[n_results++] = waited_ms;

        xSemaphoreGive(lock);
    }
}

static void ctrl_task(void *arg)
{
    (void)arg;
    vTaskDelay(pdMS_TO_TICKS(500));        // let the log settle after boot

    ESP_LOGW(TAG, "=== %s, %d trials ===",
               USE_MUTEX ? "MUTEX (inheritance)" : "BINARY SEMAPHORE", TRIALS);

    for (int i = 0; i < TRIALS; i++) {
        med_ran_during_hold = false;
        ESP_LOGI(TAG, "trial %d", i + 1);

        xTaskNotifyGive(h_low);             // start the trial
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY); // wait for LOW to finish
    }
}

```

```

        vTaskDelay(pdMS_TO_TICKS(MED_MS + 100));    // let HIGH and MED finish
    }

    // ---- aggregate ----
    int32_t mn = results[0], mx = results[0];
    int64_t sum = 0;
    for (int i = 0; i < n_results; i++) {
        if (results[i] < mn) mn = results[i];
        if (results[i] > mx) mx = results[i];
        sum += results[i];
    }

    ESP_LOGW(TAG, "=====");
    ESP_LOGW(TAG, " lock type : %s", USE_MUTEX ? "MUTEX" : "BINARY SEMAPHORE");
    ESP_LOGW(TAG, " trials      : %d", n_results);
    ESP_LOGW(TAG, " min        : %" PRIu32 " ms", mn);
    ESP_LOGW(TAG, " mean        : %" PRIu32 " ms", (int32_t)(sum / n_results));
    ESP_LOGW(TAG, " max         : %" PRIu32 " ms", mx);
    ESP_LOGW(TAG, " spread      : %" PRIu32 " ms", mx - mn);
    ESP_LOGW(TAG, "=====");

    vTaskDelete(NULL);
}

// ----- app_main
void app_main(void)
{
    #if USE_MUTEX
        // PART B -- a mutex is created ALREADY AVAILABLE. Do NOT give it here.
        lock = xSemaphoreCreateMutex();
        configASSERT(lock);
    #else
        // PART A -- a binary semaphore is created EMPTY, so it needs one give,
        // or the very first take would block forever and nothing would run.
        lock = xSemaphoreCreateBinary();
        configASSERT(lock);
        xSemaphoreGive(lock);
    #endif

    xTaskCreatePinnedToCore(low_task, "low", 3072, NULL, PRIO_LOW, &h_low, CORE);
    xTaskCreatePinnedToCore(med_task, "med", 3072, NULL, PRIO_MED, &h_med, CORE);
    xTaskCreatePinnedToCore(high_task, "high", 3072, NULL, PRIO_HIGH, &h_high, CORE);
    xTaskCreatePinnedToCore(ctrl_task, "ctrl", 3072, NULL, PRIO_CTRL, &h_ctrl, CORE);
}

```

16.6 Code walkthrough, line by line

#define USE_MUTEX 0

The single knob. Part A is 0, Part B is 1, and **nothing else in the file changes between them**. That is what makes this a controlled experiment: any difference in the measured wait is attributable to that one line.

#define CORE 0

Pins everything to a single core. Not a detail — on two cores MED could run on core 1 while LOW continued on core 0, and the inversion would be partly masked. Forcing one core makes the effect clean and reproducible.

static SemaphoreHandle_t lock;

Note the **type is the same for both primitives**. `SemaphoreHandle_t` covers a binary semaphore, a counting semaphore and a mutex; which one you have is decided entirely by the create function. That is why this bug is so easy to introduce in real code — the declaration gives you no hint.

static void busy(int ms)

`esp_timer_get_time()` returns microseconds since boot as an `int64_t` from a free-running counter, so `end` is a target timestamp and the empty `while` body spins until it is reached.

The cast `(int64_t)ms * 1000` matters. Without it, `ms * 1000` is computed in `int`, and while 500 000 fits fine today, the habit of promoting *before* multiplying is what stops the overflow bug you eventually hit with a larger value.

Everywhere else in this series a busy-wait is a bug. Here it is **required**, for the reason in the comment. Knowing when to deliberately break your own rule is worth noticing.

ulTaskNotifyTake(pdTRUE, portMAX_DELAY)

Blocks with zero CPU cost until someone notifies this task. `pdTRUE` means "clear the notification value on exit," making it behave like a binary semaphore. `portMAX_DELAY` means wait forever.

A **direct-to-task notification** is lighter than a semaphore — each task has a built-in notification value, so there is no separate kernel object to allocate. The restriction is that it targets one specific task, which is exactly what this experiment needs.

xTaskNotifyGive(h_high) immediately after LOW takes the lock

The line that guarantees the collision. HIGH is priority 8, so it preempts LOW *on this line*, attempts the lock, finds LOW holding it, and blocks. The inversion condition is now established by construction rather than by timing luck.

then xTaskNotifyGive(h_med)

Only reached because LOW resumes after HIGH blocks. MED at priority 5 becomes ready while LOW at priority 3 holds the lock — the exact configuration the concept section describes.

t_high_blocked = esp_timer_get_time(); before the take

The clock starts **before** `xSemaphoreTake`, not inside it. That is the definition of the measurement: the wait begins the instant HIGH decides it wants the lock and ends when it holds it. Timestamping after the take would measure nothing.

%" PRId32 "

From `<inttypes.h>`. The underlying type of `int32_t` varies across ESP-IDF targets and versions, so a hardcoded `%d` or `%ld` produces a format warning on some builds and wrong output on others. `PRId32` expands to whatever is correct for this platform. Use it for every fixed-width type you print.

ctrl_task at priority 9

Higher than all three participants, so it can always start the next trial and always regain control at the end. Without an orchestrator, "run it thirty times" would be thirty chances for the tasks to drift out of the intended order.

vTaskDelay(pdMS_TO_TICKS(MED_MS + 100)) between trials

LOW signals CTRL as soon as it releases the lock, but HIGH and MED are still finishing. This settle time lets them complete so trials cannot overlap. 600 ms is MED's full burn plus margin.

the #if USE_MUTEX block in app_main

WARNING

Switching to Part B is NOT a one-line change

A **binary semaphore is created empty**, so it needs the initial `xSemaphoreGive()` or the first take blocks forever. A **mutex is created already available**. Carrying that give across means giving away a mutex you never took, which FreeRTOS treats as an error and which corrupts the ownership record priority inheritance depends on — so the very mechanism you are measuring would misbehave.

That is why the two variants are written out separately behind `#if` rather than described as "swap the constructor."

configASSERT(lock)

Creation returns `NULL` on allocation failure. Without this check the first `xSemaphoreTake(NULL, ...)` crashes somewhere unrelated and the backtrace points at the wrong place. One line, and it turns a mystery into a clear abort.

16.7 Phase 2 — build, flash, run

```
idf.py build
idf.py -p COM6 erase-flash
idf.py -p COM6 flash monitor
```

Substitute your own port. `erase-flash` is worth doing once here because a stale partition map from an earlier lab is a common source of confusing behaviour.

KEY POINT

Expected output, Part A (USE_MUTEX 0)

Each trial should print this ordering — note **MED_RUN sits between HIGH_BLOCKED and LOW_UNLOCKED**:

```
I lab16: trial 1
I lab16:  LOW_LOCKED
I lab16:  HIGH_BLOCKED
I lab16:  MED_RUN           <-- MED runs while HIGH waits.  THE INVERSION.
I lab16:  LOW_RUN
I lab16:  LOW_UNLOCKED
I lab16:  HIGH_ACQUIRED after 802 ms
...

```

```

W lab16: =====
W lab16: lock type : BINARY SEMAPHORE
W lab16: trials    : 30
W lab16: min      : 800 ms
W lab16: mean     : 802 ms
W lab16: max      : 806 ms
W lab16: spread   : 6 ms
W lab16: =====

```

Now change `USE_MUTEX` to `1`, rebuild, reflash.

KEY POINT

Expected output, Part B (USE_MUTEX 1)

MED_RUN is now absent from the window between HIGH_BLOCKED and LOW_UNLOCKED.
That absence is the whole result.

```

I lab16: trial 1
I lab16:  LOW_LOCKED
I lab16:  HIGH_BLOCKED
I lab16:  LOW_RUN           <-- LOW inherited priority 8; MED cannot preempt it
I lab16:  LOW_UNLOCKED
I lab16:  HIGH_ACQUIRED after 301 ms
I lab16:  MED_RUN          <-- MED runs only AFTER the handoff

...

W lab16: lock type : MUTEX
W lab16: min      : 300 ms
W lab16: mean     : 301 ms
W lab16: max      : 304 ms
W lab16: spread   : 4 ms

```

16.8 Phase 3 — prove it

Three checks. Record all three in `debug_log.md`; they are the interview material.

#	Check	Pass condition
1	The numbers separate. Run both parts, record min/mean/max.	Semaphore near 800 ms , mutex near 300 ms . Distributions must not overlap.
2	The trace order differs. Look at where <code>MED_RUN</code> falls.	Part A: between <code>HIGH_BLOCKED</code> and <code>LOW_UNLOCKED</code> . Part B: after <code>HIGH_ACQUIRED</code> .
3	The spread is small. Check max minus min.	Under about 10 ms. A large spread means the ordering is not deterministic — check <code>CONFIG_FREERTOS_HZ</code> is 1000.

NOTE**Where the arithmetic comes from, and why hardware will not be exact**

Semaphore: MED's 500 ms plus LOW's remaining 300 ms = **800 ms**. **Mutex:** LOW's 300 ms critical section alone = **300 ms**.

Expect a few milliseconds above those figures. Tick granularity, the logging calls inside the measured window, and context-switch overhead all add a little. **What must survive is the separation**, not the exact value. If you see 802 and 301, that is a correct result, not a discrepancy.

WARNING**If the two numbers come out the same**

Almost always one of three things, in order of likelihood:

1. You left the `xSemaphoreGive` in when switching to the mutex. Check the `#if` block.
2. Tasks are not on one core. Confirm every `xTaskCreatePinnedToCore` passes `CORE`, not `tskNO_AFFINITY`.
3. Priorities are wrong. MED must be strictly between LOW and HIGH. Make MED 2 and it never preempts LOW; make it 9 and it preempts HIGH too, which is a different situation entirely.

16.9 Extensions, worked

Two things worth having, both done for you. Add them, run them, record the result — do not spend an evening deriving them.

16.9.1 — Proving the delay is *unbounded*

The strongest version of this result is not "the semaphore is slower." It is "the semaphore's delay **has no upper bound**." Add a second medium task and the semaphore wait grows while the mutex wait does not move.

Add to the globals:

```
static TaskHandle_t h_med2;
#define MED2_MS 400
#define PRI0_MED2 6           // still strictly between LOW(3) and HIGH(8)
```

Add the task:

```
static void med2_task(void *arg)
{
    (void)arg;
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        ESP_LOGI(TAG, " MED2_RUN");
        busy(MED2_MS);
    }
}
```

Notify it in `low_task`, right after the existing MED notify:

```
xTaskNotifyGive(h_med);
xTaskNotifyGive(h_med2);           // second medium-priority task
```

Create it in `app_main`, and widen the settle delay in `ctrl_task` to `MED_MS + MED2_MS + 100`:

```
xTaskCreatePinnedToCore(med2_task, "med2", 3072, NULL, PRIO_MED2, &h_med2, CORE);
```

KEY POINT

The result, and what it means

	One MED	Two MEDs	Change
Binary semaphore	~800 ms	~1200 ms	grew by MED2's full 400 ms
Mutex	~300 ms	~300 ms	did not move at all

That flat mutex row is the entire point. The semaphore's delay is a function of *how much unrelated medium-priority work exists in the system*, which is not something you can bound at design time. The mutex's delay is a function only of *how long the holder's critical section is*, which you control and can measure. **That is what "bounded" means**, and it is a much better interview answer than "the mutex was faster."

16.9.2 — What a mutex costs

The obvious follow-up question is "so why not always use a mutex?" Measure it rather than guess. Add this to `app_main` before the tasks are created:

```
static void bench_lock(void)
{
    const int N = 10000;
    int64_t t0 = esp_timer_get_time();
    for (int i = 0; i < N; i++) {
        xSemaphoreTake(lock, portMAX_DELAY);
        xSemaphoreGive(lock);
    }
    int64_t ns = (esp_timer_get_time() - t0) * 1000 / N;
    ESP_LOGW(TAG, "uncontended take+give: %" PRIu64 " ns", ns);
}
```

KEY POINT

Typical result and the conclusion

On this board, an uncontended take-plus-give runs on the order of **one to two microseconds** for either primitive, with the mutex slightly slower because it maintains the owner record.

So the honest answer is: **the overhead difference is real but tiny, and irrelevant next to an unbounded delay**. Use a mutex whenever you are protecting a resource. Reach for a semaphore or notification when you are signalling between contexts, where there is no owner to record and nothing to inherit — and note that an ISR cannot own a mutex, which is why the `FromISR` family covers semaphores and notifications but not mutexes.

16.10 Docs and references

Topic	Where
Mutexes and priority inheritance	FreeRTOS API reference, <code>xSemaphoreCreateMutex</code>
Task notifications	ESP-IDF FreeRTOS docs → Task Notifications
Tick rate configuration	<code>menuconfig</code> → Component config → FreeRTOS → Kernel → <code>configTICK_RATE_HZ</code>
The Mars Pathfinder account	Glenn Reeves, "What really happened on Mars?" — worth reading in full, it is two pages

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

NOTE

Already fixed in the lab above — nothing to do

This section is the engineering record: what the earlier version of this lab got wrong, how it was found, and what replaced it. The lab as printed above is the corrected version. It is kept because the method used to catch the error is worth more than the error itself.

Lab 16 — the experiment did not produce priority inversion

WARNING

Confirmed defect. Severity: the measurement the lab asks for cannot be obtained.

The concept, the figures and the mutex explanation are correct. **The timing values are not.** With LOW delaying 1000 ms, HIGH 1050 ms and MED 1100 ms, HIGH never blocks on the lock, because LOW has already released it 750 ms earlier.

Evidence — discrete-event simulation of the three tasks on one core with preemptive priority scheduling:

```
manual as printed (1000 / 1050 / 1100):
  binary semaphore : HIGH waits = [0, 0, 0, 0, 0] ms
  mutex           : HIGH waits = [0, 0, 0, 0, 0] ms

simulator validated against a configuration that SHOULD invert
(HIGH wakes at 100 ms, inside LOW's 300 ms hold):
  binary semaphore : HIGH waits = [1200, ...] ms
  mutex           : HIGH waits = [ 200, ...] ms
```

The first cycle runs: LOW takes the lock at $t=0$ and releases at $t=300$. HIGH wakes at $t=1050$ and finds it free. MED runs 1100–1600. LOW wakes at 1300 but is blocked by MED until 1600. From there the three periodic loops drift apart by different amounts, so any later collision is scheduler-dependent rather than designed. **That is the opposite of a controlled experiment.**

Fix: Lab 16 replaces the tuned delays with explicit task-notification ordering, which makes the sequence deterministic and repeatable. Verified result: **800 ms on every trial with a binary semaphore, 300 ms on every trial with a mutex**, with no variance across 24 trials.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 16.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16 ←	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.